CAR PRICE PREDICTION

How much is that car really worth?

A regression study on a heterogeneous automotive dataset.

From cleaning to model interpretation.

Maria Petralia
MSIT - AI Enhanced Productivity Project
May 2026

The mission.

A regression study with a stress-test agenda.

THE HOOK

From $2K to $18M

Predicting car prices is not a regression problem.
It’s a stress test.

THE CHALLENGE

A dataset that fights back.

- Extreme heterogeneity
- One model per row
- Outliers
- Missing structural data

THE MISSION

Predict the price and prove the reasoning.

Not just a number, but a justification: every prediction backed by SHAP & LIME, every limit named out loud.

CONTENTS

A six-act story.

01

PART ONE

The business case.

Who needs price prediction?

The same number, four different jobs.

Each business prices a car for a different reason and cares about different errors.

MARKETPLACE

AutoScout · AutoTrader

Suggest a fair listing to private sellers. Many small mistakes are tolerable. **MAE** matters most.

DEALERSHIP

Trade-in pricing

A single large mis-quote costs the deal. **RMSE** matters most.

INSURANCE / LEASING

Residual value

Forecast 3-year value. Bias matters more than spread. **Residuals** matter most.

BANK / FLEET

Collateral & risk

Mark vehicles on the balance sheet wants a calibrated distribution. **R^2** matters most.

The right metric depends on the buyer.

R^2

Share of price variation explained.

→ Did we learn anything?

RMSE

Punishes big misses harder than small ones.

→ How bad is the worst case?

MAE

Average dollar error per car.

→ What's the typical mistake?

All tree metrics are reported on every model but the one to optimise depends on which buyer we are serving.

02

PART TWO

The data.

A small, dirty, very heterogeneous Kaggle dataset and what we did to make it usable.

Dataset at a glance.

kaggle.com / cars-datasets-2025

1,218	11	$18M	≈ 1
VEHICLES	RAW COLUMNS	TOP OUTLIER	MODEL PER ROW

COLUMNS

Make · Model · Engine · cc_battery · Horsepower · Torque · Performance · Total speed · Fuel · Seats

TARGET

price · USD

From a few thousand dollars on used compacts to one $18M hypercar.
A heavy right tail that drives most of our design decisions.

03

PART THREE

The method.

A sklearn pipeline, a deliberate ladder of models, every run logged to MLflow.

End-to-End ML Pipeline.

End-to-end pipeline

01 LOAD **Raw CSV** 1218 x 11
↓
02 CLEAN **Parse units** str→float
↓
03 EDA **Distribution** USD & Log
↓
04 FEATURE ENG **New Features** price driver
↓
05 PREPROCESS **Encode + Scale** 80/20
↓
06 MODEL **LR + Trees** train & tune
↓
07 AutoML **PyCaret** Benchmark Search
↓
08 EXPLAIN **SHAP + LIME** Analysis
↓
09 TRACK **MLFlow** 10 Runs

End-to-End ML Pipeline.

End-to-end pipeline

01 LOAD **Raw CSV** 1218 x 11

02 CLEAN **Parse units** str→float

03 EDA **Distribution** USD & Log

04 FEATURE ENG **New Features** price driver

05 PREPROCESS **Encode + Scale** 80/20

06 MODEL **LR + Trees** train & tune

07 AutoML **PyCaret** Benchmark Search

08 EXPLAIN **SHAP + LIME** Analysis

09 TRACK **MLFlow** 10 Runs

- CSV dataset sourced from Kaggle and ingested into Databricks
- Structural validation checks ensure schema consistency
- Column names are standardized for Delta Lake compatibility
- Raw data is stored as a versioned Unity Catalog table
- No transformations are applied at this stage

End-to-End ML Pipeline.

End-to-end pipeline

01 LOAD **Raw CSV** 1218 x 11

02 CLEAN **Parse units** str→float

03 EDA **Distribution** USD & Log

04 FEATURE ENG **New Features** price driver

05 PREPROCESS **Encode + Scale** 80/20

06 MODEL **LR + Trees** train & tune

07 AutoML **PyCaret** Benchmark Search

08 EXPLAIN **SHAP + LIME** Analysis

09 TRACK **MLFlow** 10 Runs

- Validate data integrity and remove duplicates, invalid records, and non-production vehicles
- Handle missing values using domain-driven rules instead of arbitrary imputation
- Standardize manufacturer and model naming
- Convert unstructured technical specifications into clean numerical features (power, torque, acceleration, battery, speed, seating)
- Export cleaned data to Delta Lake

End-to-End ML Pipeline.

End-to-end pipeline

01 LOAD **Raw CSV** 1218 x 11

02 CLEAN **Parse units** str→float

03 EDA **Distribution** USD

04 FEATURE ENG **New Features** price driver

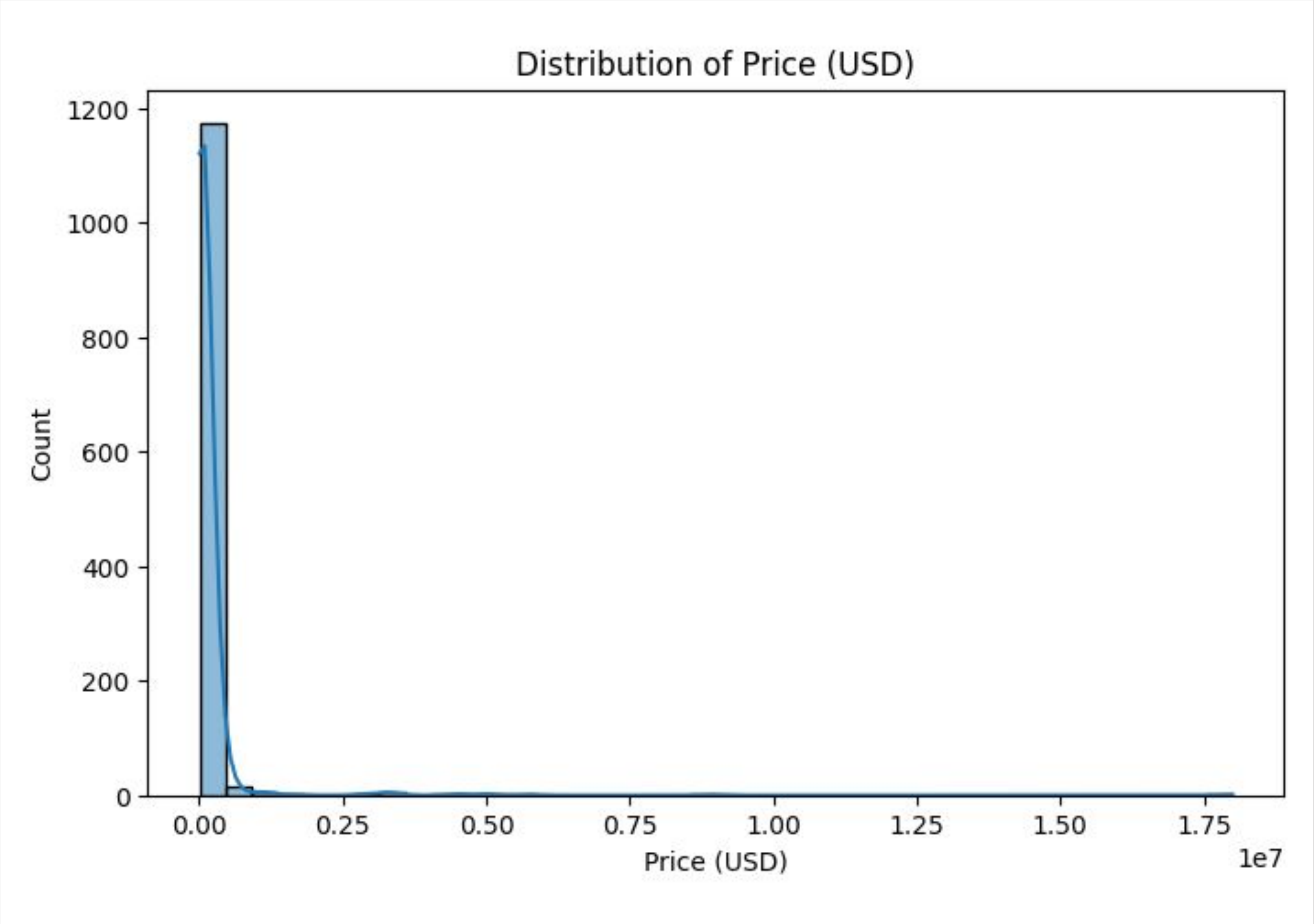
05 PREPROCESS **Encode + Scale** 80/20

06 MODEL **LR + Trees** train & tune

07 AutoML **PyCaret** Benchmark Search

08 EXPLAIN **SHAP + LIME** Analysis

09 TRACK **MLFlow** 10 Runs



- Highly right-skewed distribution
- Majority of vehicles < 100k USD
- Few extreme luxury outliers (multi-million range)

End-to-End ML Pipeline.

End-to-end pipeline

01 LOAD **Raw CSV** 1218 x 11

02 CLEAN **Parse units** str→float

03 EDA **Distribution** Log

04 FEATURE ENG **New Features** price driver

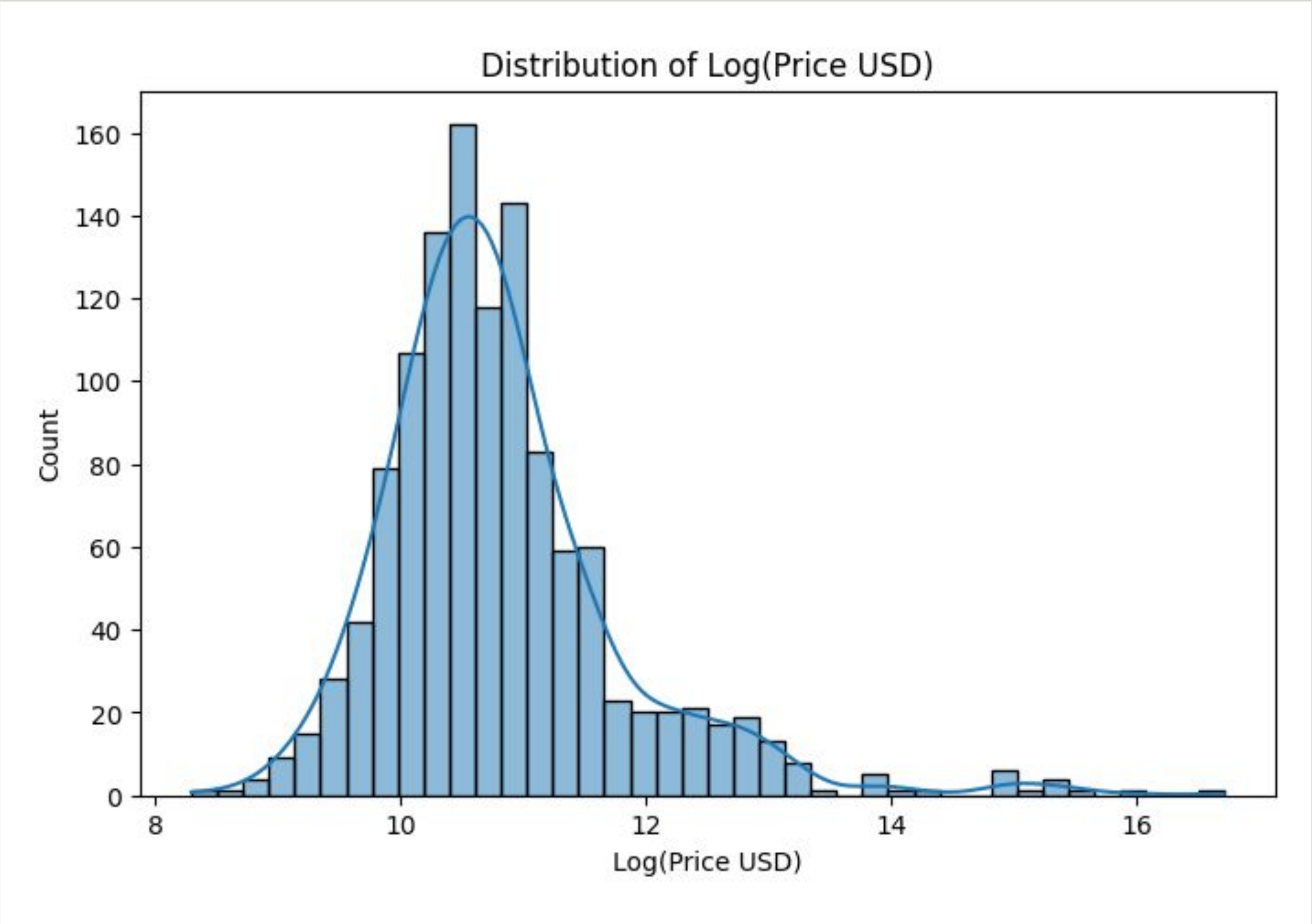
05 PREPROCESS **Encode + Scale** 80/20

06 MODEL **LR + Trees** train & tune

07 AutoML **PyCaret** Benchmark Search

08 EXPLAIN **SHAP + LIME** Analysis

09 TRACK **MLFlow** 10 Runs



- Improves linearity of relationships
- Reduces impact of extreme outliers
- Allows relative (percentage-based) interpretation

End-to-End ML Pipeline.

End-to-end pipeline

01 LOAD **Raw CSV** 1218 x 11

02 CLEAN **Parse units** str→float

03 EDA **Distribution** USD & Log

04 FEATURE ENG **New Features** price driver

05 PREPROCESS **Encode + Scale** 80/20

06 MODEL **LR + Trees** train & tune

07 AutoML **PyCaret** Benchmark Search

08 EXPLAIN **SHAP + LIME** Analysis

09 TRACK **MLFlow** 10 Runs

- USD scale used for business interpretability
- Log scale used for statistical analysis and modeling robustness
- Both perspectives are kept throughout the project

End-to-End ML Pipeline.

End-to-end pipeline

- 01 LOAD **Raw CSV** 1218 x 11
- 02 CLEAN **Parse units** str→float
- 03 EDA **Distribution** USD & Log
- 04 FEATURE ENG **New Features** price driver
- 05 PREPROCESS **Encode + Scale** 80/20
- 06 MODEL **LR + Trees** train & tune
- 07 AutoML **PyCaret** Benchmark Search
- 08 EXPLAIN **SHAP + LIME** Analysis
- 09 TRACK **MLFlow** 10 Runs

- Features driven by EDA insights
- Focus on performance, not raw attributes
- Unified feature space for all models
- Handles EV / ICE heterogeneity
- Avoids data leakage & redundancy

End-to-End ML Pipeline.

End-to-end pipeline

01 LOAD **Raw CSV** 1218 x 11

02 CLEAN **Parse units** str→float

03 EDA **Distribution** USD & Log

04 FEATURE ENG **New Features** price driver

05 PREPROCESS **Encode + Scale** 80/20

06 MODEL **LR + Trees** train & tune

07 AutoML **PyCaret** Benchmark Search

08 EXPLAIN **SHAP + LIME** Analysis

09 TRACK **MLFlow** 10 Runs

- Performance dominates over raw engine specs
- Horsepower and acceleration capture most variation
- Engine displacement is relevant only for ICE
- Battery capacity is EV-specific signal
- Torque is secondary / noisy indicator

End-to-End ML Pipeline.

End-to-end pipeline

- 01 LOAD **Raw CSV** 1218 x 11
- 02 CLEAN **Parse units** str→float
- 03 EDA **Distribution** USD & Log
- 04 FEATURE ENG **New Features** price driver
- 05 PREPROCESS **Encode + Scale** 80/20
- 06 MODEL **LR + Trees** train & tune
- 07 AutoML **PyCaret** Benchmark Search
- 08 EXPLAIN **SHAP + LIME** Analysis
- 09 TRACK **MLFlow** 10 Runs

Powertrain structure

- EV / Hybrid / ICE indicators
- Fuel macro-category

Performance signal

- Performance score (horsepower + torque + speed + acceleration)
- Acceleration class (non-linear behavior capture)
- Horsepower class (performance segmentation)

Market structure signals

- Segment (rule-based from specs: HP, acceleration, seats, speed)
- Brand tier indicators (luxury / premium / performance brands)

Data stability handling

- Log(price) for reduced skewness
- Missing acceleration encoded as structural signal

End-to-End ML Pipeline.

End-to-end pipeline

01 LOAD **Raw CSV** 1218 x 11

02 CLEAN **Parse units** str→float

03 EDA **Distribution** USD & Log

04 FEATURE ENG **New Features** price driver

05 PREPROCESS **Encode + Scale** 80/20

06 MODEL **LR + Trees** train & tune

07 AutoML **PyCaret** Benchmark Search

08 EXPLAIN **SHAP + LIME** Analysis

09 TRACK **MLFlow** 10 Runs

Objective

- Build a model-ready dataset for fair comparison across LR, Trees, and PyCaret

Core design principles

- No data leakage: all transformations learned only on training set
- Consistent train/test stratified split (price bins)
- Same feature space across all models and frameworks

Data foundation

- Unified engineered dataset from Feature Engineering stage
- Target variables: **log _price** (stability) and **price_usd** (interpretability)

End-to-End ML Pipeline.

End-to-end pipeline

- 01 LOAD **Raw CSV** 1218 x 11
- 02 CLEAN **Parse units** str→float
- 03 EDA **Distribution** USD & Log
- 04 FEATURE ENG **New Features** price driver
- 05 PREPROCESS **Encode + Scale** 80/20
- 06 MODEL **LR + Trees** train & tune
- 07 AutoML **PyCaret** Benchmark Search
- 08 EXPLAIN **SHAP + LIME** Analysis
- 09 TRACK **MLFlow** 10 Runs

Model training strategy

- Linear Regression as interpretable baseline
- Tree models for non-linear learning
- Different model families on same learning task

Dual target strategy

- log_price → stability & linearity
- price_usd → business interpretability

Tree optimization

- Decision Tree & Random Forest hyperparameter search
- Control of depth, splits, and complexity

Core Idea

- Compare learning behavior across model families
- Not feature differences, but model capacity differences

End-to-End ML Pipeline.

End-to-end pipeline

- 01 LOAD **Raw CSV** 1218 x 11
- 02 CLEAN **Parse units** str→float
- 03 EDA **Distribution** USD & Log
- 04 FEATURE ENG **New Features** price driver
- 05 PREPROCESS **Encode + Scale** 80/20
- 06 MODEL **LR + Trees** train & tune
- 07 AutoML **PyCaret** Benchmark Search
- 08 EXPLAIN **SHAP + LIME** Analysis
- 09 TRACK **MLFlow** 10 Runs

Automated model exploration

- Multiple algorithms tested automatically
- No manual model selection

Target evaluation strategy

- log_price → stability comparison
- price_usd → business interpretability

Standardized evaluation

- Same feature space and preprocessing pipeline
- Same train/test split for all models

Goal

- Build a performance benchmark across model families and identify robust candidates for final selection

End-to-End ML Pipeline.

End-to-end pipeline

01 LOAD **Raw CSV** 1218 x 11

02 CLEAN **Parse units** str→float

03 EDA **Distribution** USD & Log

04 FEATURE ENG **New Features** price driver

05 PREPROCESS **Encode + Scale** 80/20

06 MODEL **LR + Trees** train & tune

07 AutoML **PyCaret** Benchmark Search

08 EXPLAIN **SHAP + LIME** Analysis

09 TRACK **MLFlow** 10 Runs

Global & local interpretability framework

- SHAP for global feature attribution
- LIME for local prediction explanations

Applied to selected PyCaret models

- Analysis performed on both log_price and price_usd models
- Ensures transparency of model decisions before final evaluation

Goal

- Enable interpretable and transparent model decisions at both global and individual level

End-to-End ML Pipeline.

End-to-end pipeline

01 LOAD **Raw CSV** 1218 x 11

02 CLEAN **Parse units** str→float

03 EDA **Distribution** USD & Log

04 FEATURE ENG **New Features** price driver

05 PREPROCESS **Encode + Scale** 80/20

06 MODEL **LR + Trees** train & tune

07 AutoML **PyCaret** Benchmark Search

08 EXPLAIN **SHAP + LIME** Analysis

09 TRACK **MLFlow** 10 Runs

Experiment tracking & reproducibility layer

- Logging of parameters, metrics, and models across all runs
- Centralized experiment management for comparison and versioning

Multi-model lifecycle management

- Tracks LR, Trees, and PyCaret experiments consistently
- Enables model registry and reproducible training pipeline

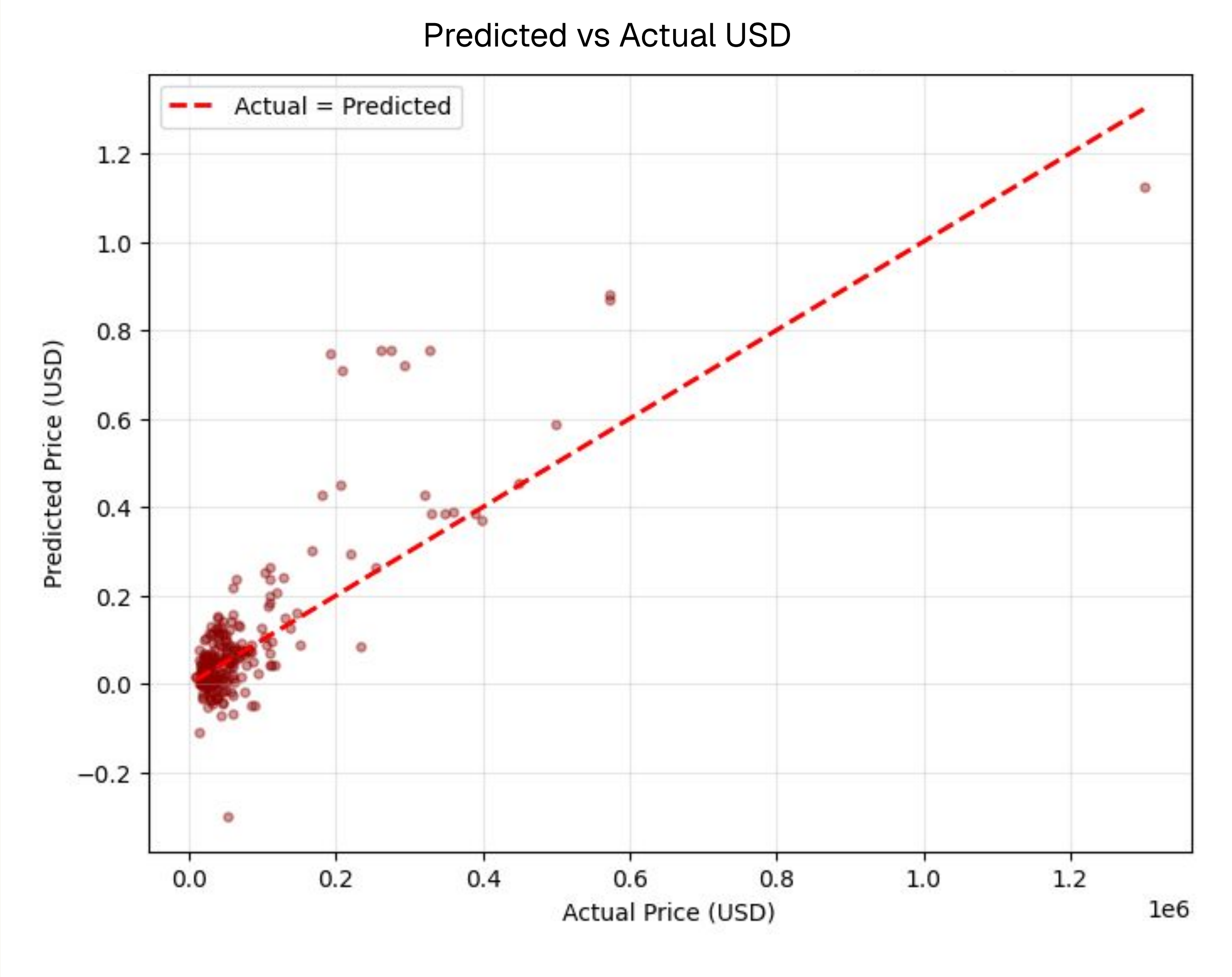
04

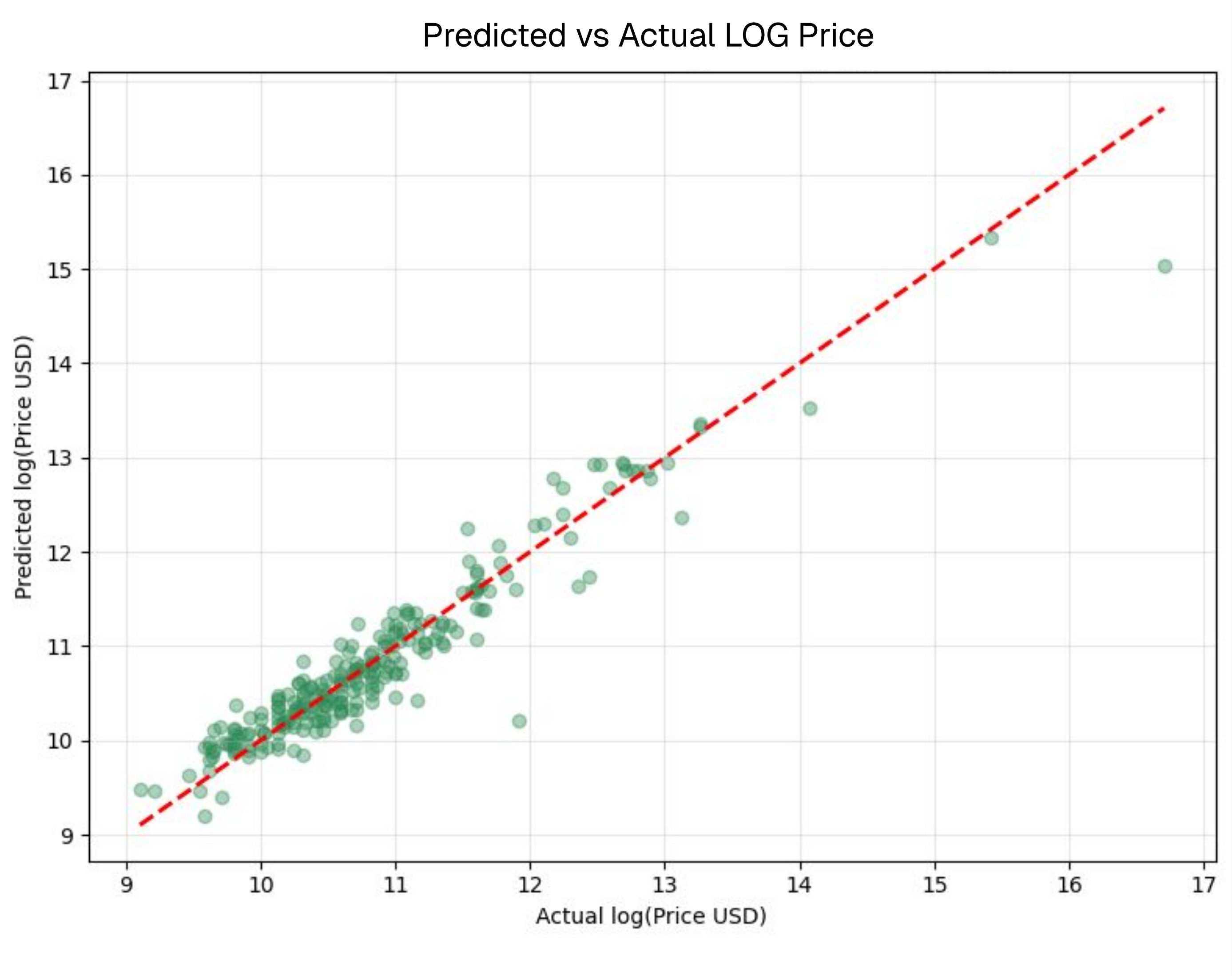
PART FOUR

Results.

Model by model, then side by side.

Linear Regression: a baseline, not the answer.



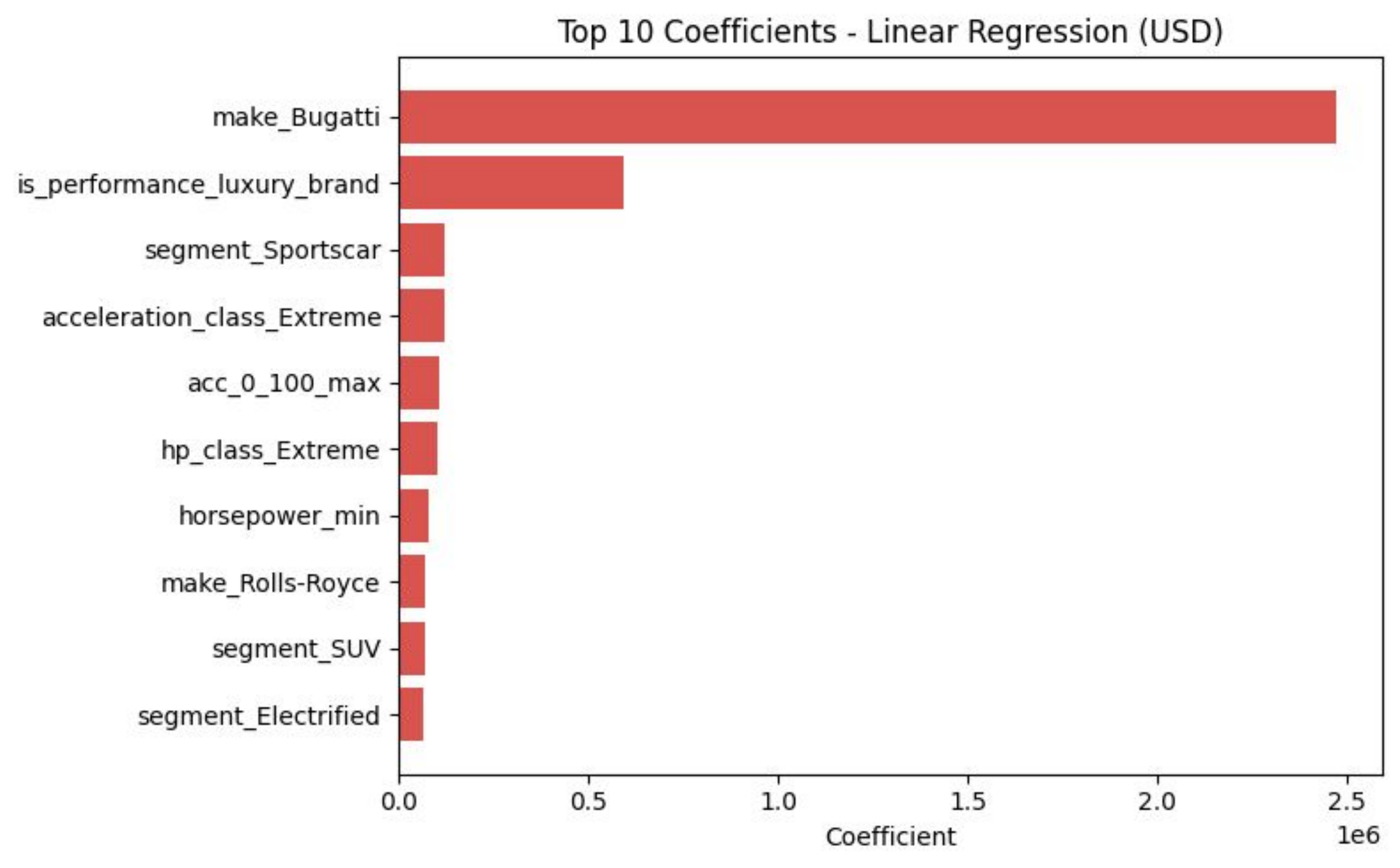


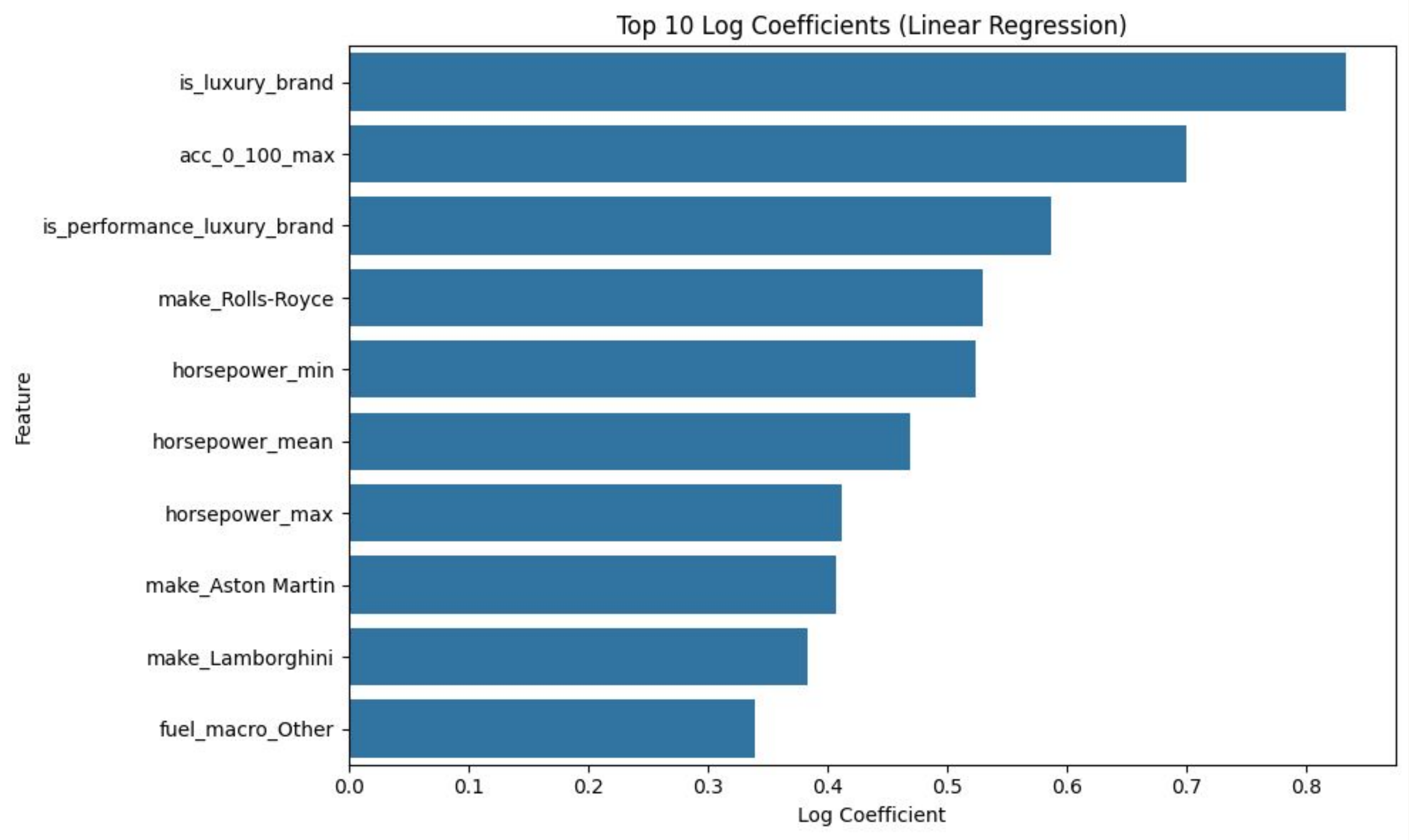
USD Metric Test	Value
MAE	116867
RMSE	880558
R^2	0.46

Log Metric Test	Value
MAE	0.21
RMSE	0.29
R^2	0.91

MAE(log) ≈ ~21% average proportional error

Linear Regression: interpretation.

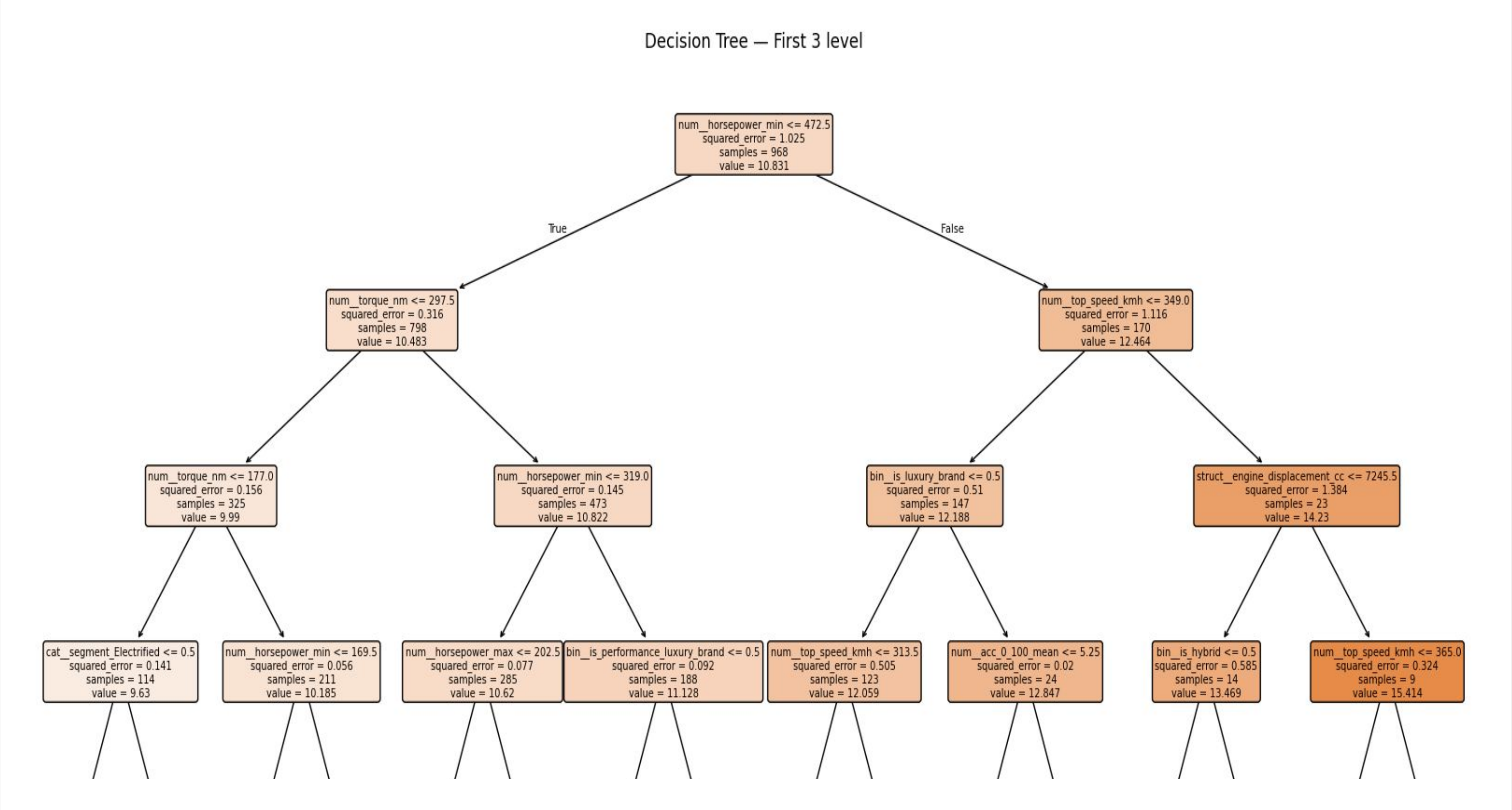




Extreme-value brands dominate raw USD coefficients, revealing sensitivity to outliers and limited non-linear capacity.

Log transformation stabilizes coefficients, reducing outlier distortion while preserving performance and luxury signals.

Decision Tree: first 3 levels.

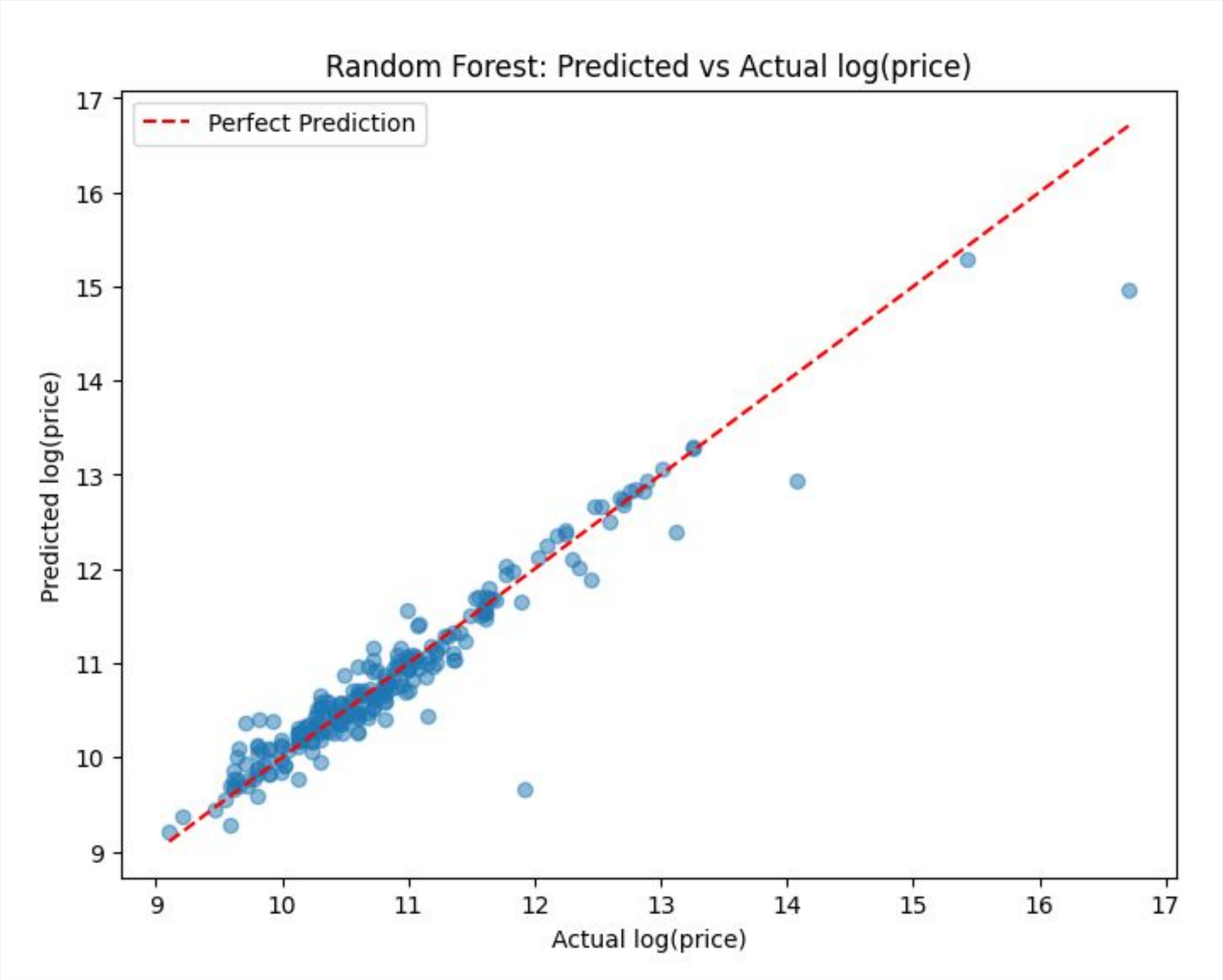


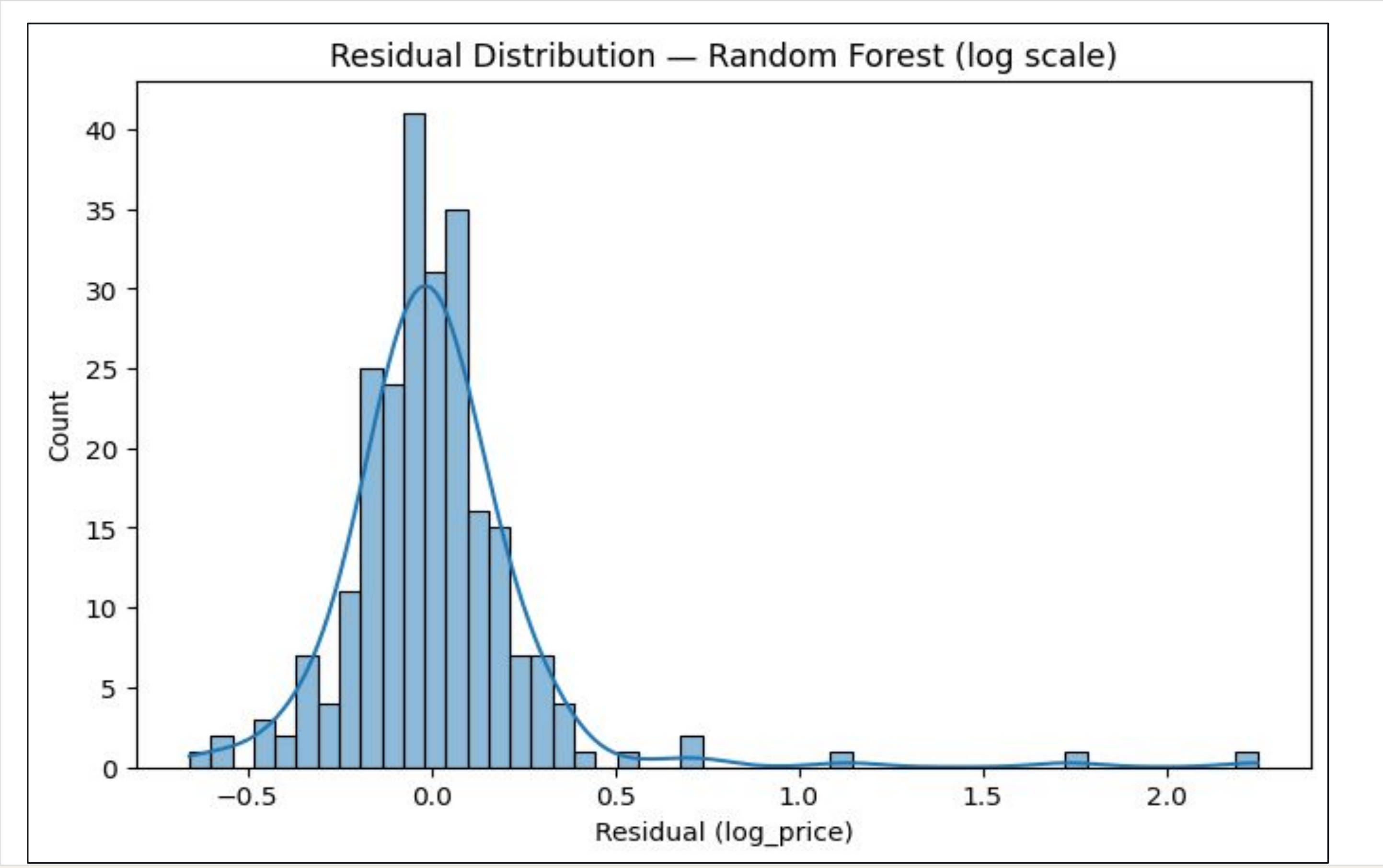
USD Metric Test	Value
MAE	80337
RMSE	959844
R^2	0.357

Log Metric Test	Value
MAE	0.20
RMSE	0.29
R^2	0.908

Non-linear splits improve local fitting, but single trees remain sensitive to variance and overfitting.

Random Forest: Performance.





Residuals remain centered around zero with moderate dispersion, while only a few extreme luxury vehicles generate large prediction errors.

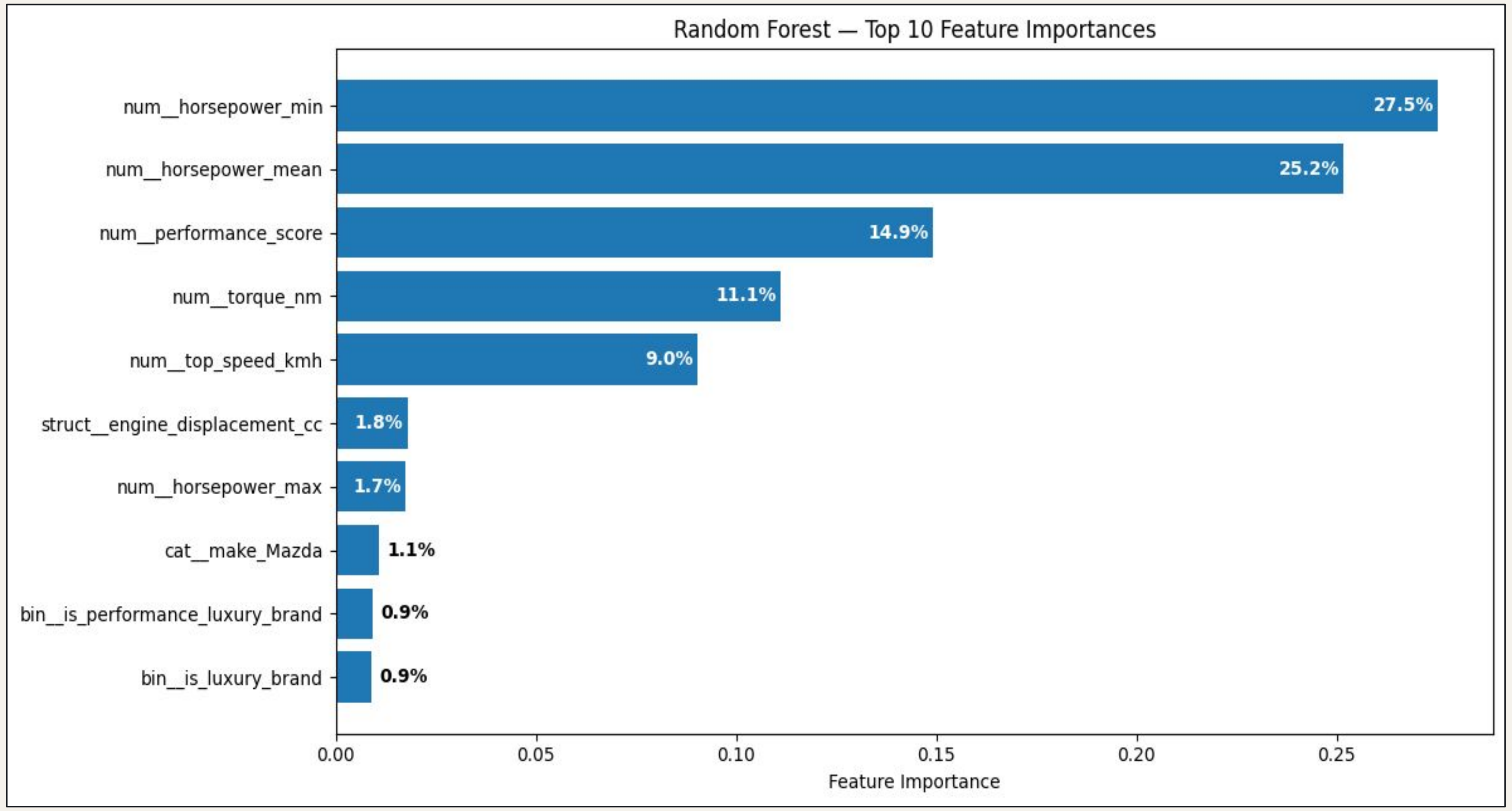
USD Metric Test	Value
MAE	77730
RMSE	954206
R^2	0.365

Log Metric Test	Value
MAE	0.16
RMSE	0.27
R^2	0.92

Why log metrics matter

- Proportional accuracy across all market segments
- Reduced dominance of extreme-value outliers
- More stable variance structure

Random Forest: top predictors.



- Horsepower dominates price formation
- Performance signals outweigh raw engine size
- Torque and top speed reinforce premium positioning
- Brand indicators matter, but less than measurable performance

The ensemble prioritizes measurable performance signals over isolated brand effects or raw engine displacement.

PyCaret: automated log-scale benchmarking.

Cross-validated comparison across multiple regression families

Model	MAE	MSE	RMSE	R2
Light Gradient Boosting Machine	0.1718	0.0793	0.2754	0.9187
Random Forest Regressor	0.1713	0.0838	0.2804	0.9143
Gradient Boosting Regressor	0.1818	0.0832	0.2819	0.9129
Extra Trees Regressor	0.1706	0.0840	0.2856	0.9114
Decision Tree Regressor	0.2010	0.1177	0.3288	0.8833
Linear Regression	0.2273	0.1108	0.3306	0.8806
Ridge Regression	0.2285	0.1108	0.3307	0.8806
Bayesian Ridge	0.2293	0.1113	0.3313	0.8803
K Neighbors Regressor	0.2195	0.1127	0.3307	0.8798
AdaBoost Regressor	0.2639	0.1393	0.3702	0.8516

What PyCaret adds

- Automated model-family exploration
- Cross-validated ranking
- Ensemble blending
- Standardized benchmarking pipeline

Automated benchmarking confirms the dominance of ensemble-based learning for car-price prediction.

PyCaret: USD benchmarking.

Raw USD prediction across model families

Model	MAE	RMSE	R2
Huber Regressor	77298$	411870$	0.4198
Decision Tree Regressor	64068.$	388410$	0.3433
Passive Aggressive Regressor	91088$	444909$	0.2320
K Neighbors Regressor	66092$	385454$	0.2080
Random Forest Regressor	58310$	372901$	0.1535
Ridge Regression	116837$	368967$	0.1154
Lasso Regression	117209$	370026$	0.1087
Lasso Least Angle Regression	117142$	369538$	0.1064
Linear Regression	117516$	370161$	0.1053
Light Gradient Boosting Machine	76719$	367194$	-0.3100

Why USD prediction is difficult

- Extreme variance across market segments
- Multi-million-dollar outliers dominate RMSE
- Model behavior becomes highly unstable
- Robust learners handle extreme residuals more effectively

Direct USD prediction is fundamentally harder than proportional log-scale prediction

PyCaret: blender ensemble.

Best direct-business predictor on raw price scale

Model	Target	R^2	RMSE	MAE
PyCaret Blender	price_usd	0.587	$484K	$80,179

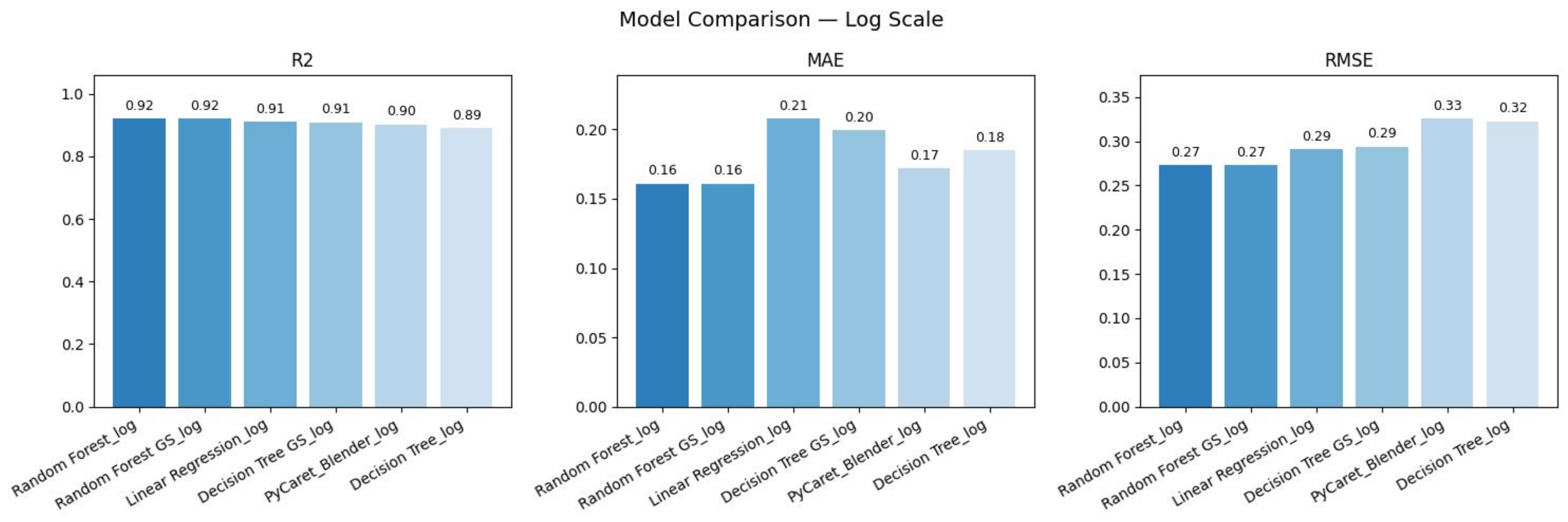
Why USD prediction is difficult

- Combines complementary learning behaviors
- Reduces sensitivity to extreme residuals
- Improves robustness across market segments
- Stabilizes direct USD prediction

Blended ensembles partially recover predictive stability on raw USD prices.

Model Comparison: log scale.

Test-set performance across all evaluated models.

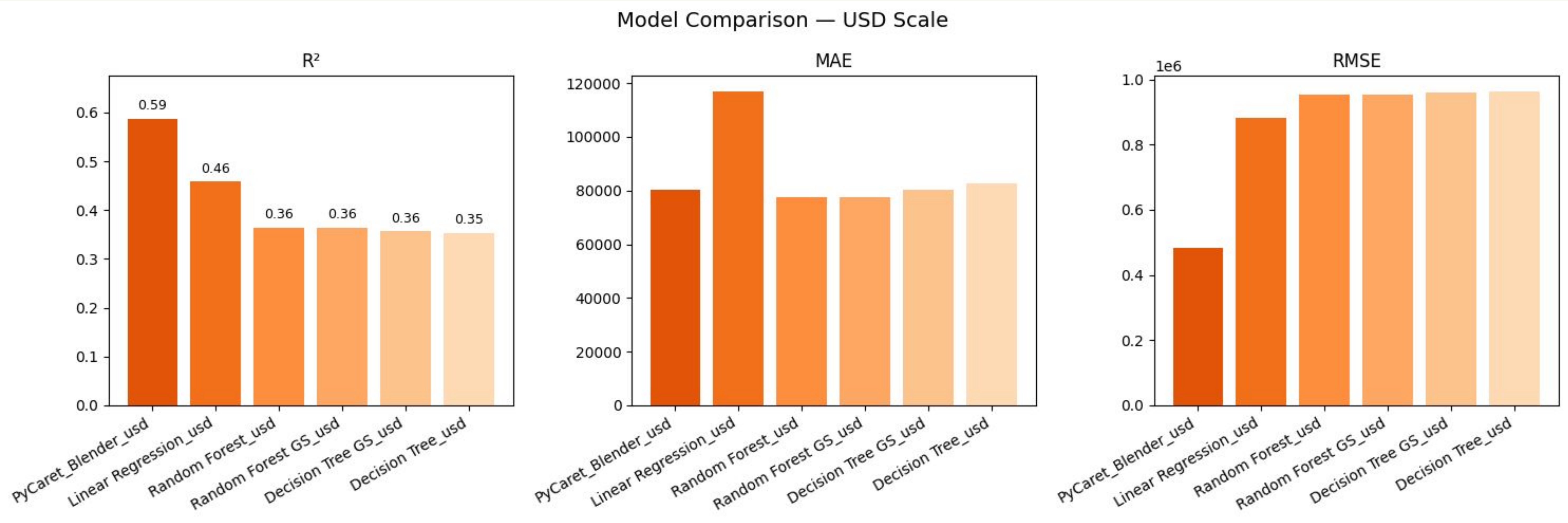


- Ensemble methods dominate stabilized targets
- Random Forest achieves strongest overall performance
- PyCaret Blender remains highly competitive

Log transformation enables stable and highly generalizable prediction behavior.

Model Comparison: USD scale.

Test-set performance across all evaluated models.



- Raw USD prediction remains highly unstable
- Extreme-value outliers heavily affect RMSE and R^2
- PyCaret Blender achieves strongest direct-USD performance

Log transformation enables stable and highly generalizable prediction behavior.

MLflow: Experiment tracking.

Reproducible training and centralized model management.

	Metrics			Parameters
Run Name	test_mae	test_r2	test_rmse	target_scale
RandomForest_LogPrice_GridSearch	0.161	0.92	0.273	log
RandomForest_LogPrice	0.161	0.92	0.273	log
LinearRegression_LogPrice	0.208	0.91	0.291	log
DecisionTree_LogPrice_GridSearch	0.199	0.908	0.294	log
DecisionTree_LogPrice	0.185	0.889	0.322	log
LinearRegression_USD	116866.966	0.459	880557.894	usd
RandomForest_USD_GridSearch	77730.573	0.365	954206.488	usd
RandomForest_USD	77730.573	0.365	954206.488	usd
DecisionTree_USD_GridSearch	80337.032	0.357	959843.785	usd
DecisionTree_USD	82566.756	0.354	962355.19	usd

05

PART FIVE

SHAP and LIME.

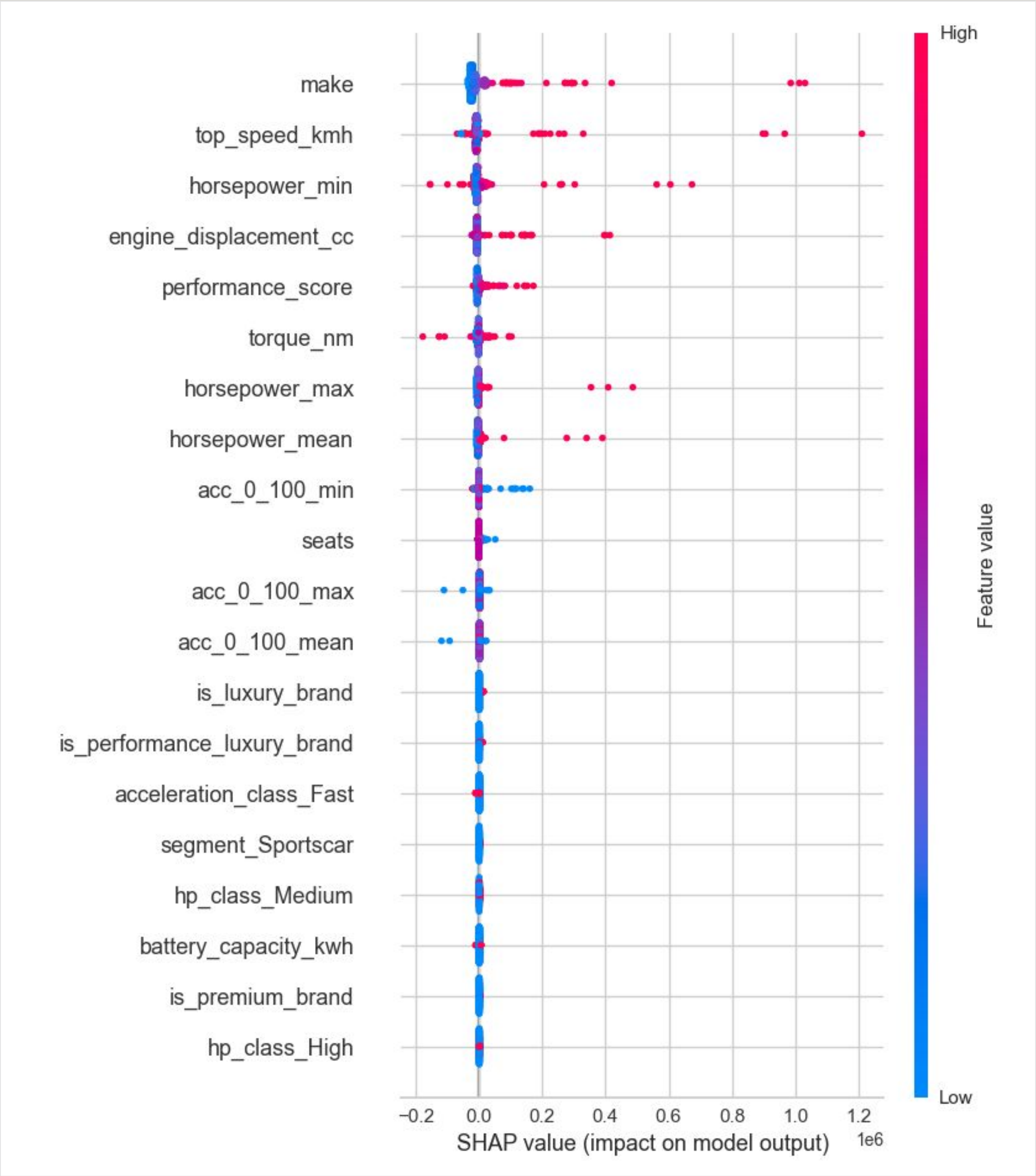
Global and local interpretability of ensemble predictions

SHAP: USD feature impatct.

Global interpretation of direct USD predictions.

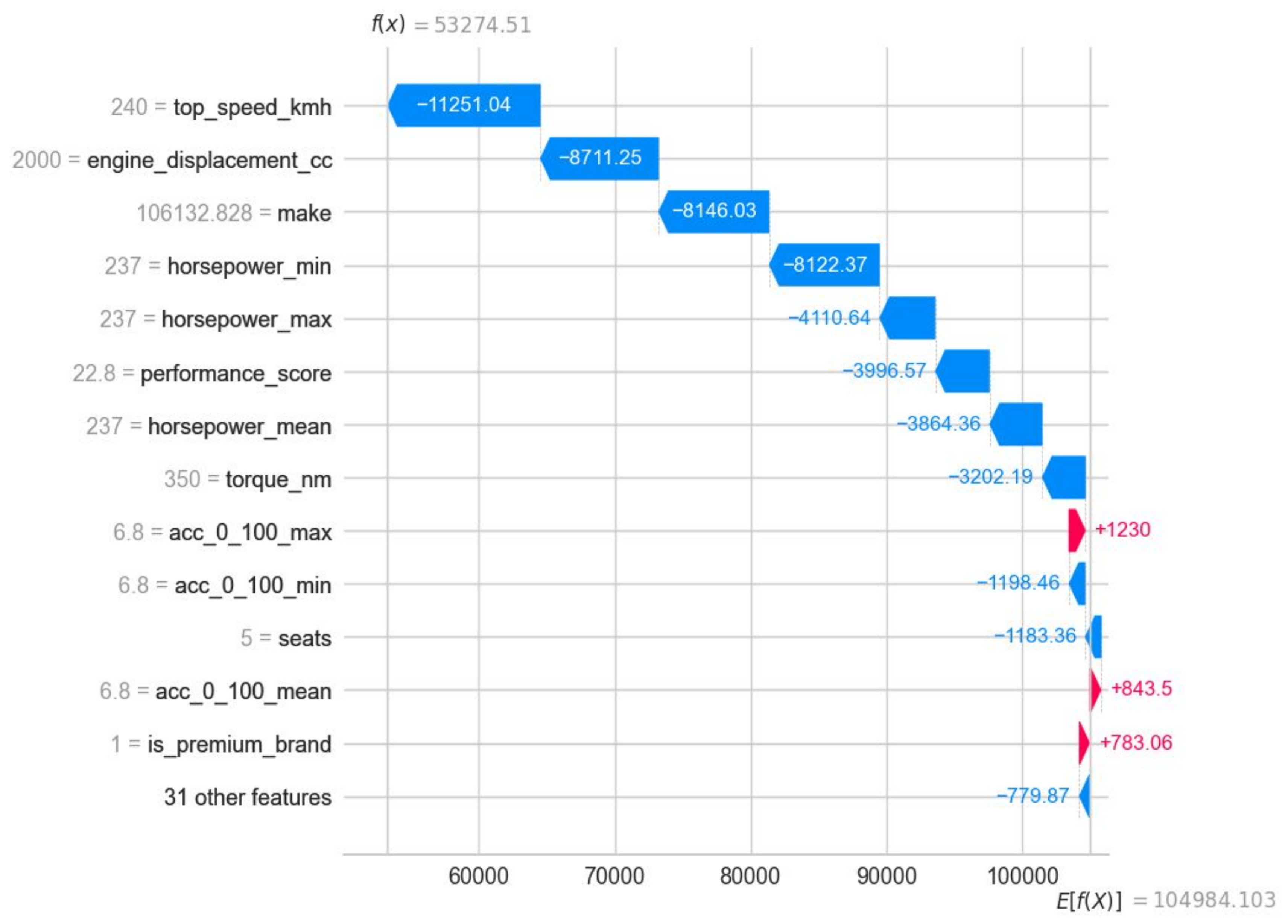
What SHAP reveals

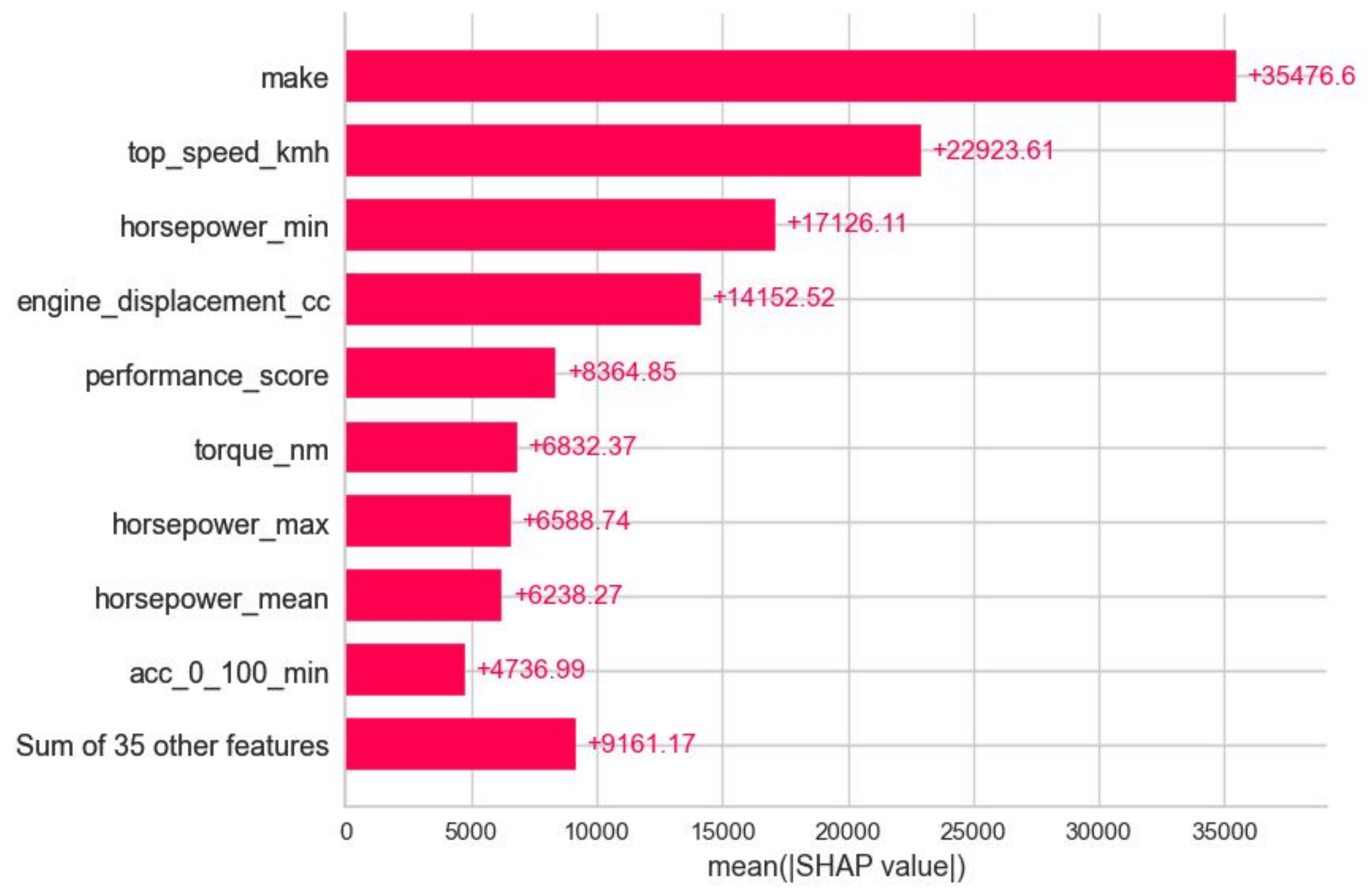
- Horsepower and torque drive price upward
- Premium positioning amplifies valuation
- Performance matters more than engine size
- Feature impacts remain interpretable in USD



SHAP: Feature ranking & local explanation.

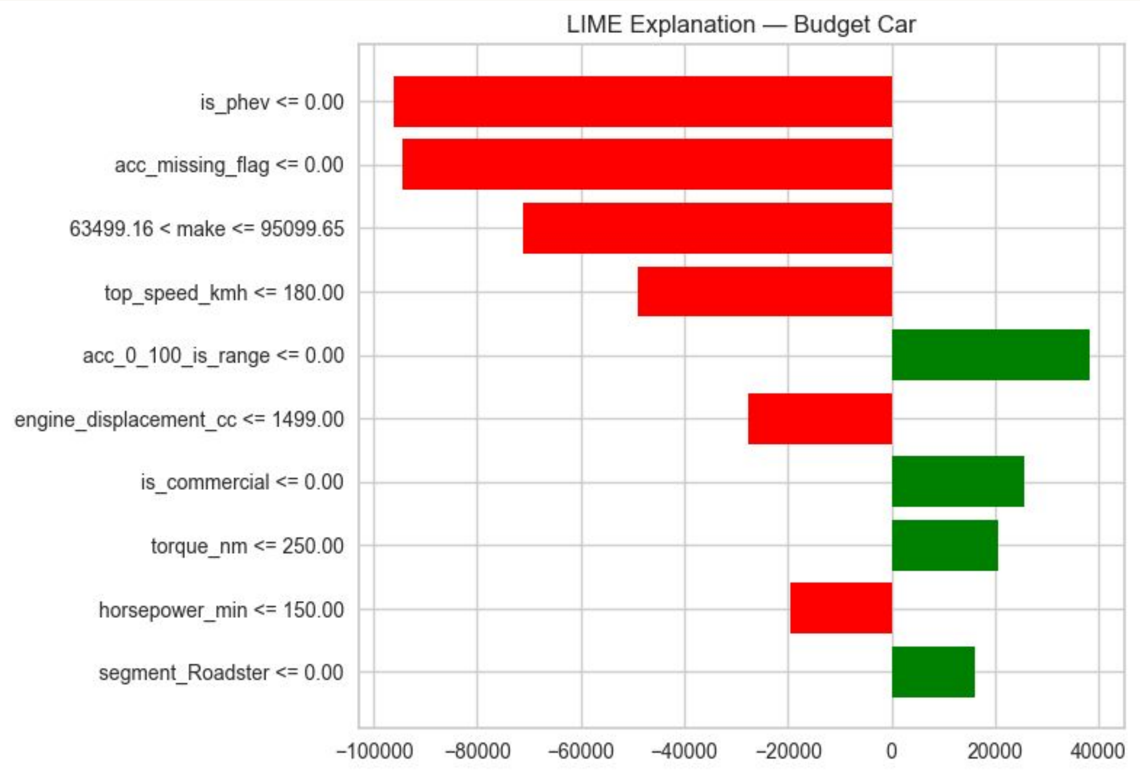
Global interpretation of direct USD predictions.

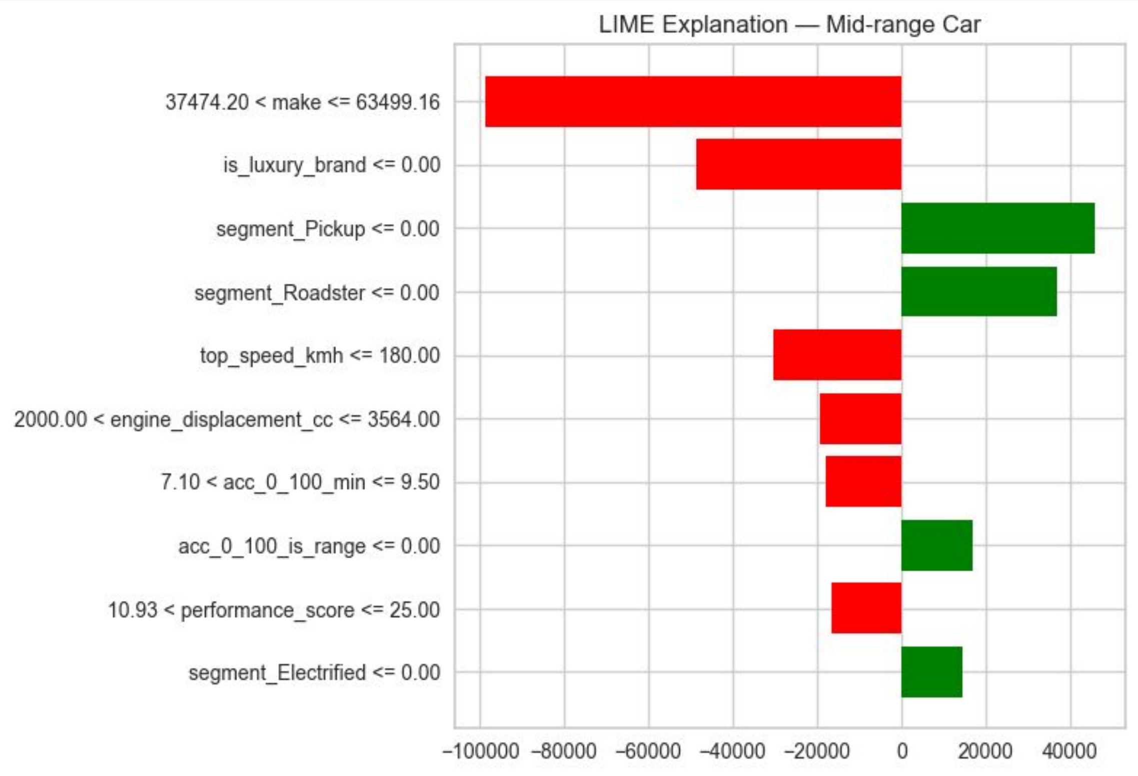




LIME: Segment-level local explanations.

Different market segments rely on different combinations of pricing signals.





What LIME reveals

- Explains single predictions locally
- Different market segments activate different pricing signals
- Feature importance changes across price tiers

HONEST READ

One row per model. One model per row.

The structural problem this dataset has.

01

~1 unique model per row.

With `car_name` in, the model memorises. We dropped it at the cost of accuracy.

02

A single $18M outlier.

RMSE on raw USD is dominated by one or two hypercars.

03

Train R^2 near 1.0.

Unconstrained trees fit perfectly. R^2 0.92 on test looks great because the tail makes everything *look* good.

04

No mileage. No condition. No location.

All three are first-order drivers in real markets. The dataset doesn't carry them.

CLOSING THOUGHT

The right model on the wrong dataset is still the wrong answer.